Activation Functions in Deep Learning

Lecture notes: what they are, why they matter, and how to choose one

Learning Objectives

By the end of this lecture, you should be able to:

- Explain why neural networks need non-linear activation functions.
- Write the formula and derivative for the major activation functions.
- Identify the vanishing-gradient and dying-neuron problems and which activations cause or fix them.
- Choose an appropriate activation function for a hidden layer and an output layer, given a task.

1. Why Do We Need Activation Functions?

A neural network layer computes a weighted sum of its inputs: $z = W \cdot x + b$. If we stacked layers using only this linear operation, the entire network — no matter how many layers — would collapse mathematically into a single linear function of the input. Composing linear functions just gives another linear function.

An activation function f is applied to z after every layer (except usually the last) to introduce non-linearity: $a = f(z)$. This non-linearity is what allows a deep network to approximate complex, non-linear relationships — curved decision boundaries, image features, language structure, and so on. Without it, a 100-layer network would be no more powerful than a single linear layer.

1.1 What makes a good activation function?

- Non-linear — otherwise depth adds no representational power.
- Differentiable (almost everywhere) — gradient descent needs a gradient to backpropagate.
- Computationally cheap — it's evaluated (and differentiated) millions/billions of times during training.
- Keeps gradients well-behaved across many layers — neither vanishing to ~0 nor exploding.
- Zero-centered output (desirable but not essential) — helps gradient descent converge more directly.

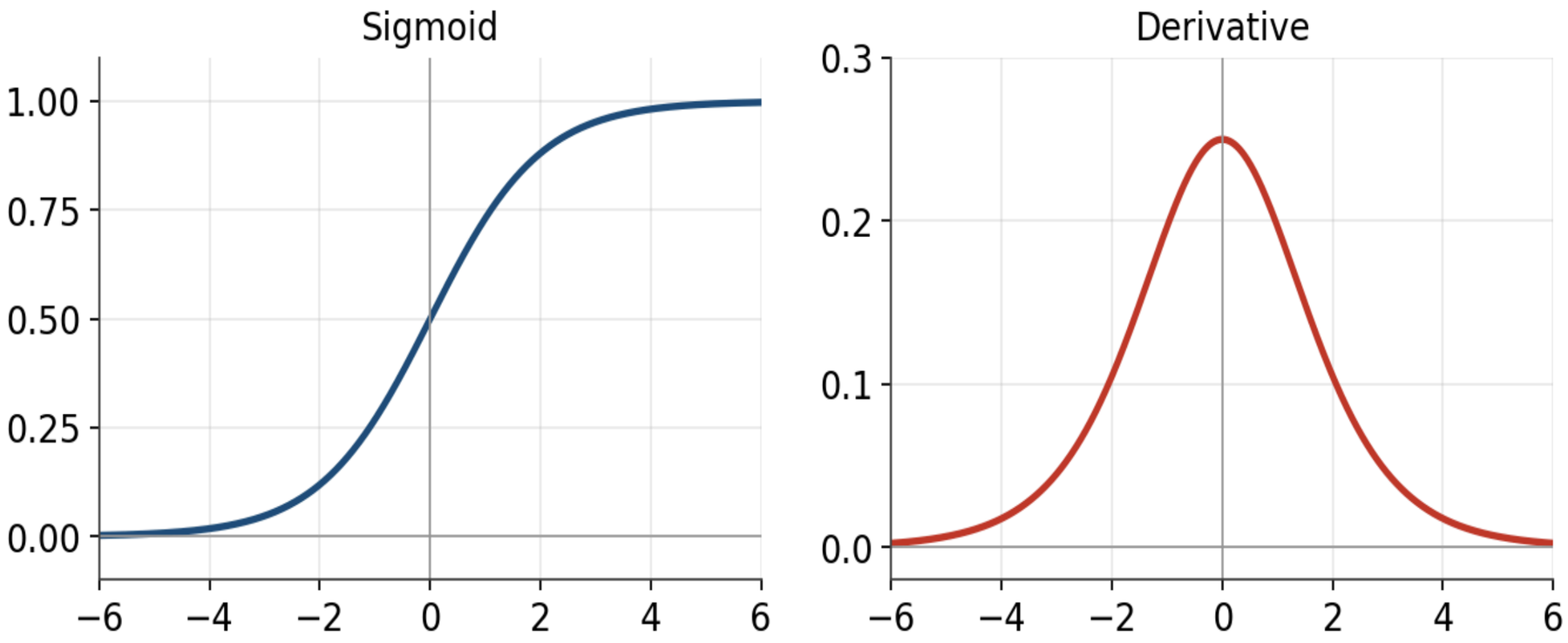
2. Classic Saturating Activations

These were the default choice in early neural networks. Both squash their input into a bounded range, which causes gradients to vanish for large |z| — their main drawback in deep networks.

2.1 Sigmoid (Logistic)

$$\sigma(z) = 1 / (1 + e^{-z})$$

Range: (0, 1)



Derivative: $\sigma(z) \times (1 - \sigma(z))$ — maximum value 0.25, at z = 0

Strengths

- Smooth, interpretable as a probability — natural for binary classification outputs.
- Historically important; still standard for output-layer binary classification and for gates in LSTMs/GRUs.

Weaknesses

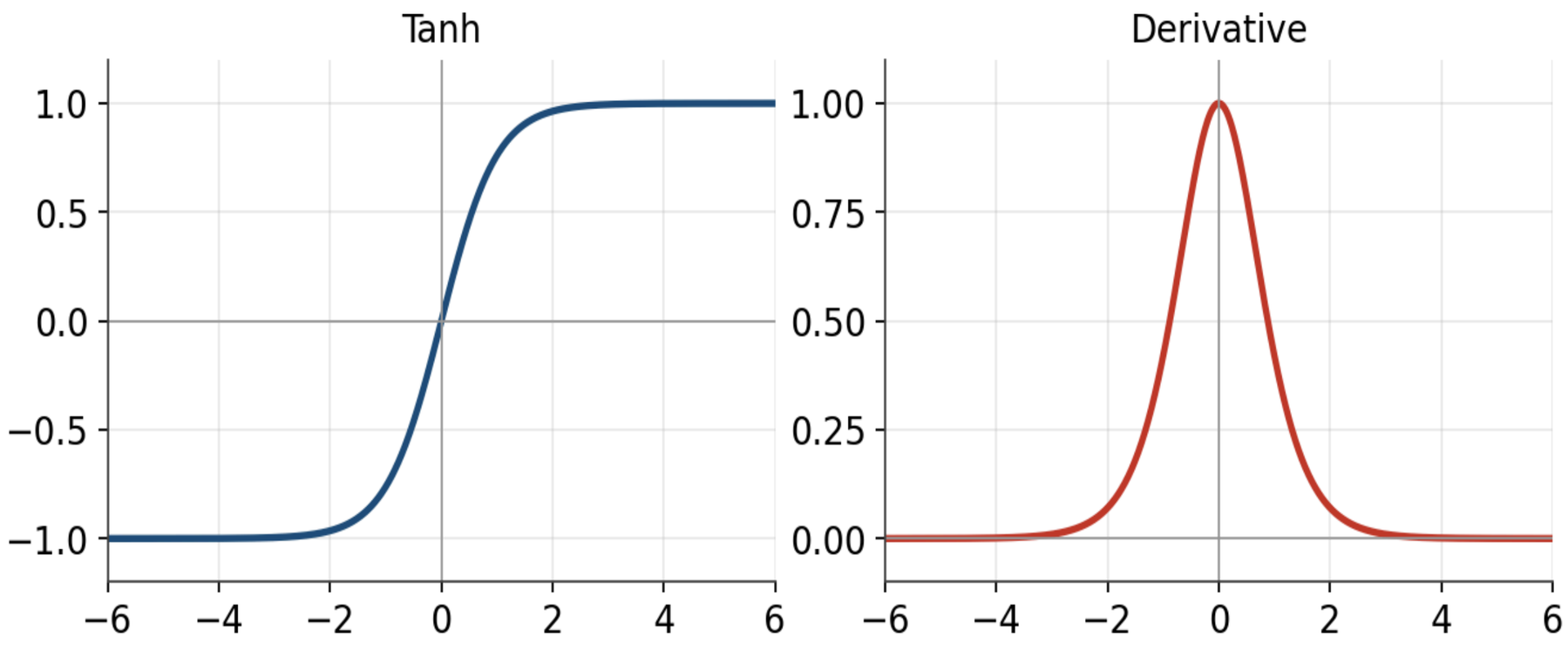
- Vanishing gradients: for |z| > ~5, the derivative is nearly 0, so earlier layers get almost no learning signal.
- Output is not zero-centered (always positive), which can slow convergence — every gradient w.r.t. the weights of a neuron has the same sign.
- Involves an exponential — relatively expensive to compute at scale.

Typically used in: output layer for binary classification; gates inside LSTM/GRU recurrent units.

2.2 Tanh (Hyperbolic Tangent)

$$\tanh(z) = (e^{z} - e^{-z}) / (e^{z} + e^{-z}) = 2\sigma(2z) - 1$$

Range: (− 1, 1)



Derivative: $1 - \tanh^2(z)$ — maximum value 1, at $z = 0$

Strengths

- Zero-centered output — usually converges faster than sigmoid in hidden layers.
- Steeper gradient near 0 than sigmoid, so learning is somewhat stronger for small activations.

Weaknesses

- Still saturates for large |z| — vanishing gradients remain a problem in deep networks.
- Still relatively expensive to compute (two exponentials).

Typically used in: hidden layers of shallow networks and RNNs (the classic 'tanh layer' in vanilla RNNs and inside LSTM cells).

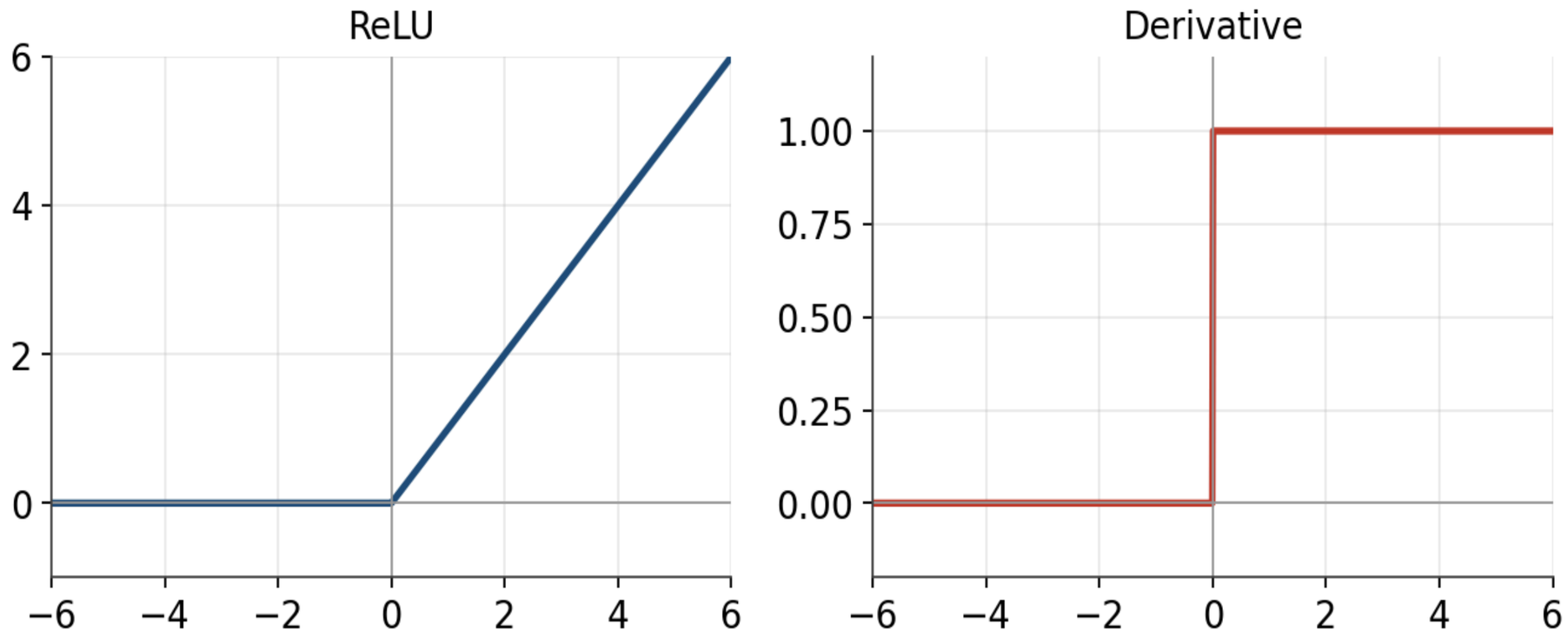
3. The ReLU Family

ReLU (Rectified Linear Unit) and its variants are the default choice for hidden layers in most modern feed-forward and convolutional networks. They don't saturate for positive inputs, which largely solves the vanishing-gradient problem and is computationally trivial.

3.1 ReLU

$$f(z) = \max(0, z)$$

Range: $[0, \infty)$



Derivative: 1 if $z > 0$, else 0 (undefined at $z = 0$, conventionally set to 0 or 1)

Strengths

- Extremely cheap to compute — just a threshold at zero.
- No saturation for $z > 0$, so gradients don't vanish on the active side — much faster training in deep networks.
- Induces sparsity: on average about half the neurons in a layer output exactly 0, which can act as a mild regularizer.

Weaknesses

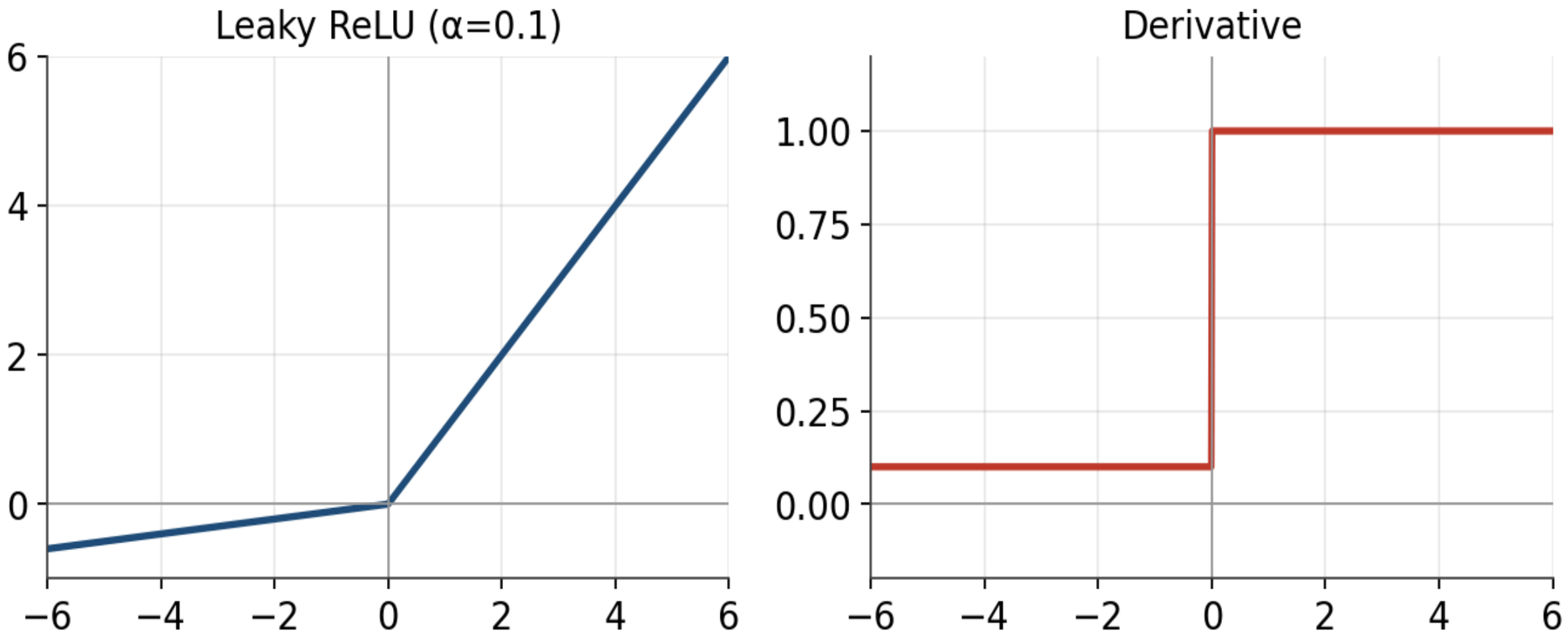
- 'Dying ReLU' problem: if a neuron's weights update such that z is always negative for all training examples, its gradient is permanently 0 and it stops learning — it's 'dead' for good.
- Not zero-centered.
- Unbounded output can occasionally contribute to exploding activations in poorly-initialized networks.

Typically used in: hidden layers of CNNs, MLPs, and most feed-forward architectures — the default first choice.

3.2 Leaky ReLU

$$f(z) = z \text{ if } z > 0, \text{ else } \alpha z \quad (\alpha \text{ is a small fixed constant, e.g. } 0.01 – 0.1)$$

Range: $(-\infty, \infty)$



Derivative: 1 if $z > 0$, else α

Strengths

- Fixes dying ReLU: negative inputs still produce a small, non-zero gradient (α), so a neuron can recover.
- Just as cheap to compute as ReLU.

Weaknesses

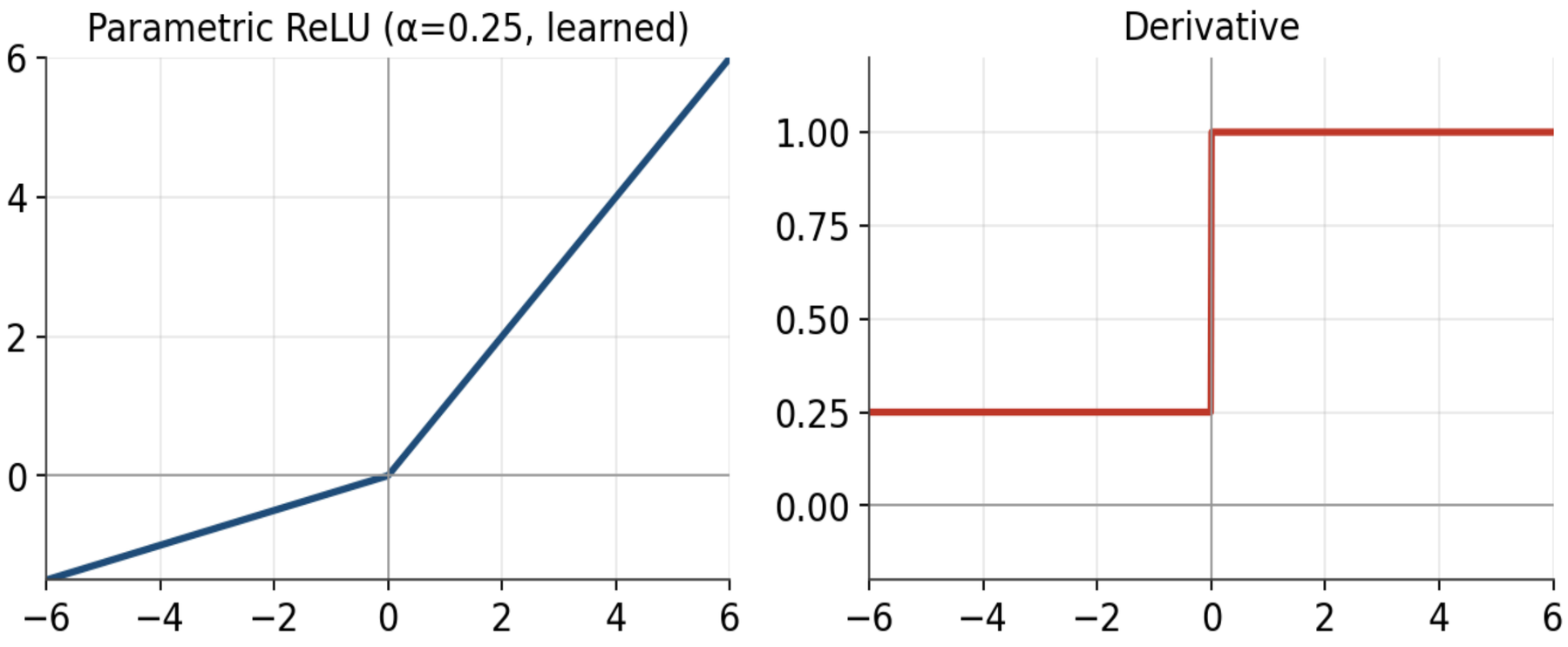
- α is a hyperparameter you must choose (or tune) — results are somewhat sensitive to it.
- Not zero-centered; benefit over plain ReLU is inconsistent across tasks.

Typically used in: as a drop-in replacement for ReLU when dead neurons are observed during training.

3.3 Parametric ReLU (PReLU)

$$f(z) = z \text{ if } z > 0, \text{ else } \alpha z \quad (\alpha \text{ is now a learned parameter, per-channel or shared})$$

Range: $(-\infty, \infty)$



Derivative: 1 if $z > 0$, else α (α updated by gradient descent like any other weight)

Strengths

- Lets the network learn the best slope for negative inputs rather than fixing it in advance.

- Small number of extra parameters — negligible compute/memory overhead.

Weaknesses

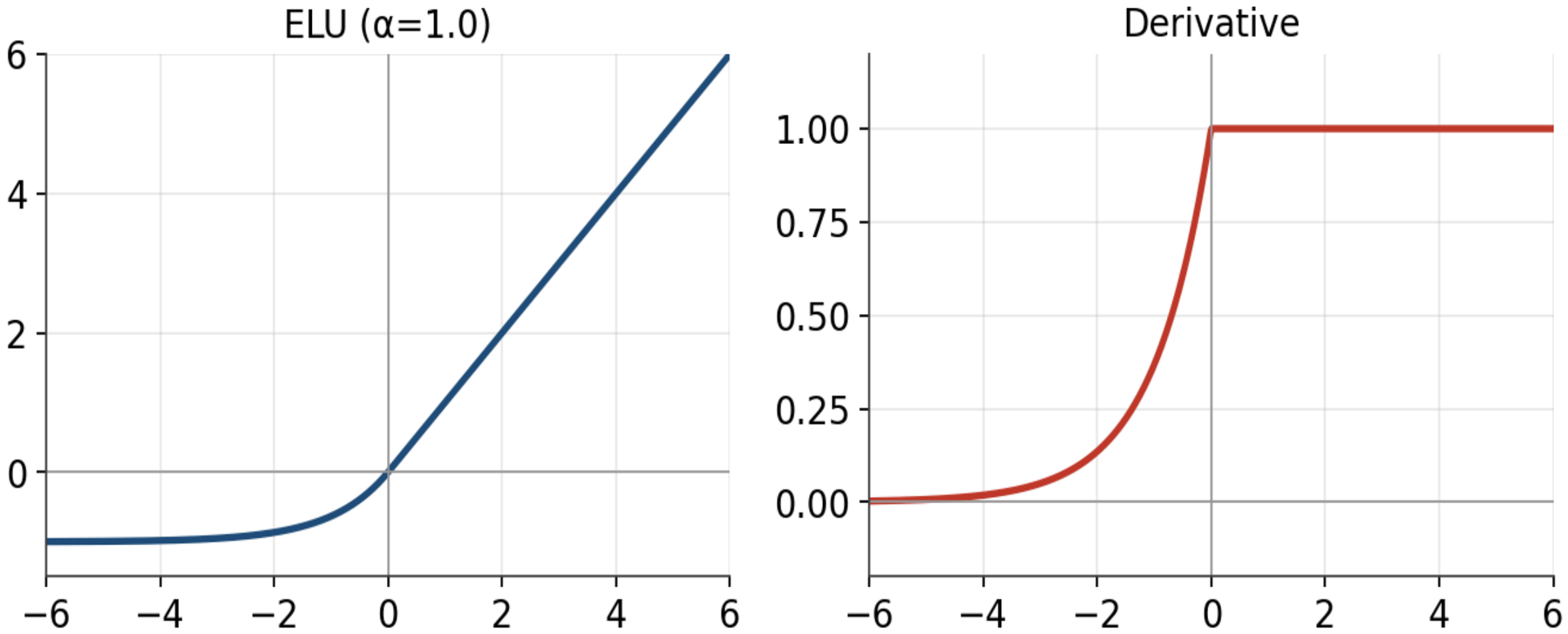
- Adds a (small) risk of overfitting, especially on small datasets.
- Slightly more complex to implement/tune than Leaky ReLU.

Typically used in: large-scale CNNs (introduced for ImageNet-scale image classification).

3.4 ELU (Exponential Linear Unit)

$$f(z) = z \text{ if } z > 0, \text{ else } \alpha(e^z - 1) \quad (\alpha \text{ typically } 1.0)$$

Range: $(-\alpha, \infty)$



Derivative: $1 \text{ if } z > 0, \text{ else } f(z) + \alpha$

Strengths

- Negative values saturate smoothly to $-\alpha$ instead of a hard zero, pushing the mean activation closer to zero — this tends to speed up learning versus ReLU.
- No dying-neuron problem — negative-side gradient is never exactly 0 for finite z.

Weaknesses

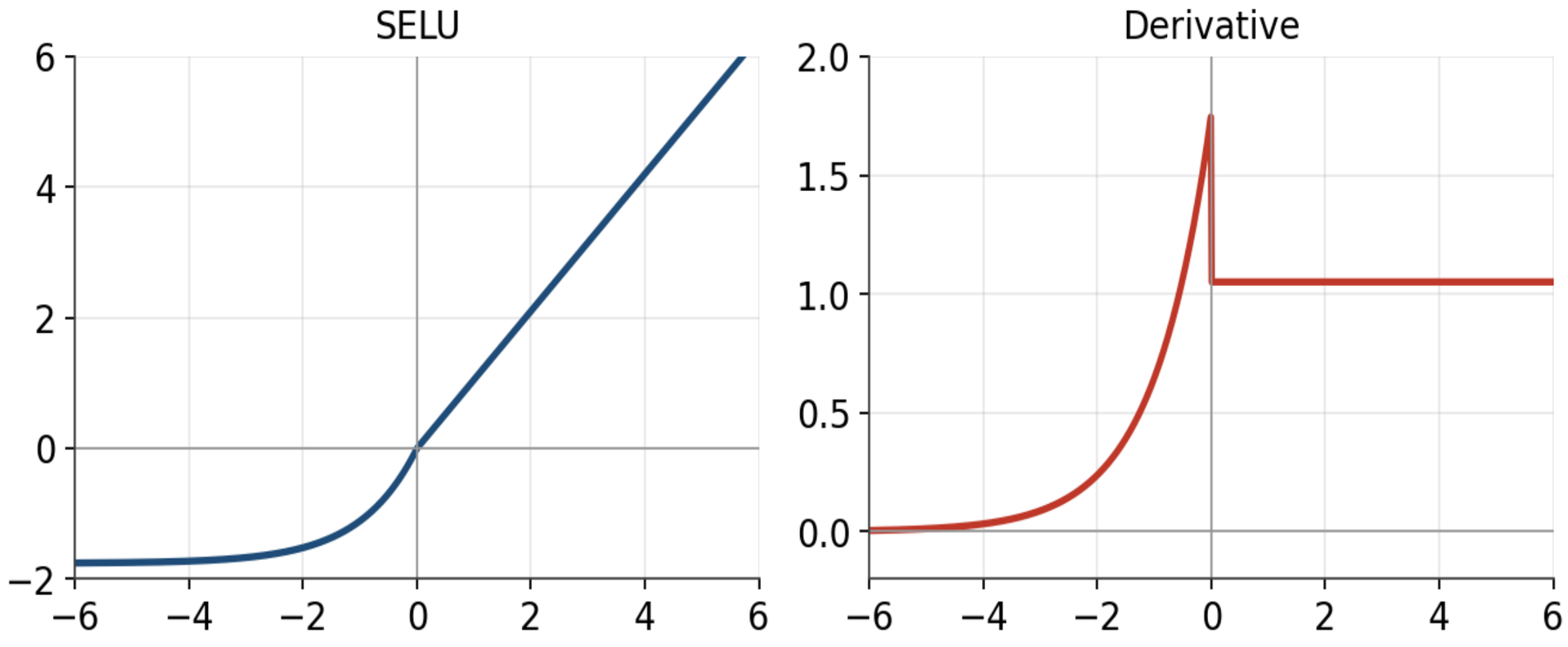
- Involves an exponential for $z < 0$ — more expensive than ReLU/Leaky ReLU.
- Introduces a hyperparameter α.

Typically used in: hidden layers where ReLU's dead-neuron issue or non-zero-centered output is hurting convergence.

3.5 SELU (Scaled Exponential Linear Unit)

$$f(z) = \lambda z \text{ if } z > 0, \text{else} \lambda\alpha(e^z - 1) \quad (\lambda \approx 1.0507, \alpha \approx 1.6733 — \text{fixed, derived constants})$$

Range: $(-\lambda\alpha, \infty)$



Derivative: λ *if* $z > 0$*, else* $\lambda\alpha e^z$

Strengths

- Self-normalizing: with the specific λ and α constants (and matching weight initialization), activations across layers automatically converge toward mean 0 / variance 1, which can remove the need for batch normalization.

Weaknesses

- Guarantees only hold under specific conditions: 'lecun_normal' weight initialization and a plain feed-forward (dense) architecture — the self-normalizing property breaks with skip connections, batch norm, dropout (needs a special 'alpha dropout'), or convolutions in some settings.
- Not widely used in CNNs/Transformers, where batch/layer normalization dominates instead.

Typically used in: deep, fully-connected ('vanilla' MLP) networks trained without batch normalization.

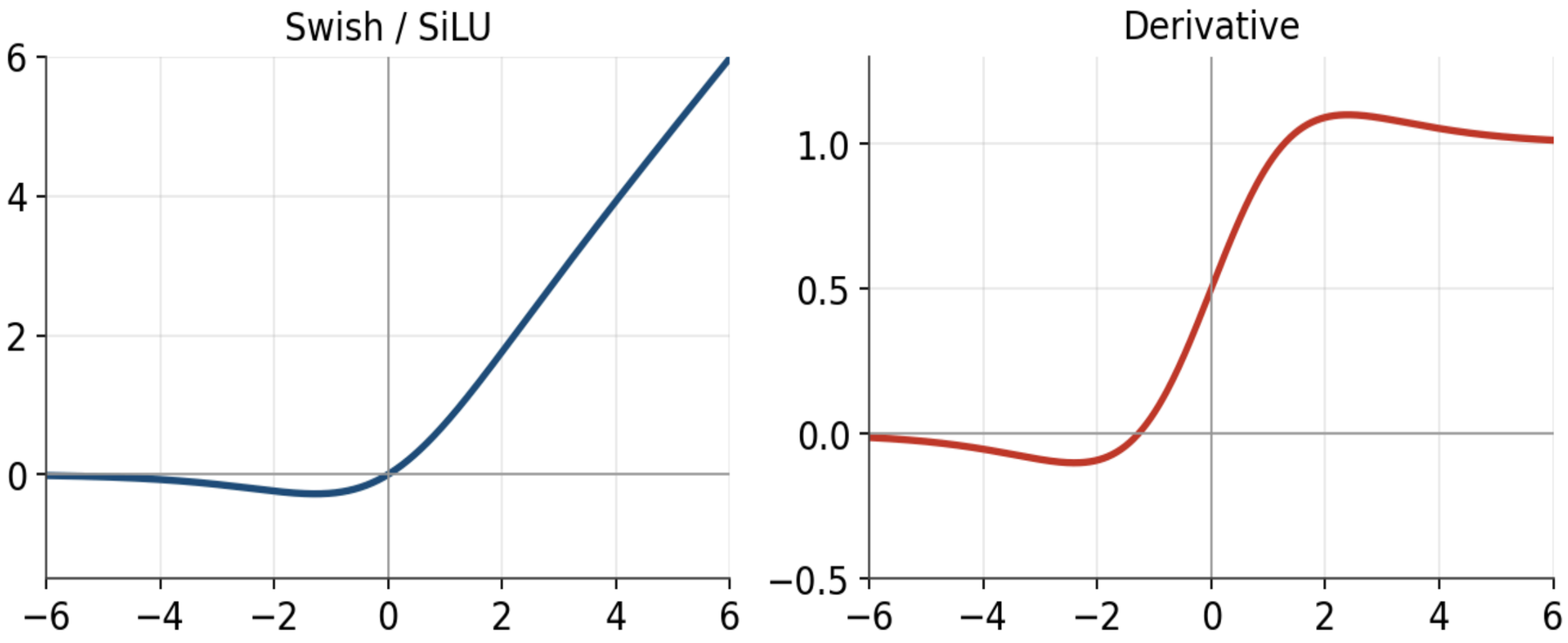
4. Smooth, Modern Activations

These functions were discovered (in some cases by automated search) to consistently outperform ReLU in very deep networks, particularly Transformers. They are smooth everywhere — including near z = 0 — which produces better-behaved gradients than ReLU's sharp corner.

4.1 Swish / SiLU (Sigmoid-weighted Linear Unit)

$$f(z) = z \times \sigma(z) \quad (\sigma = \text{sigmoid}; \beta = 1 \text{ in the common SiLU variant})$$

Range: $\approx (-0.28, \infty)$



Derivative: $\sigma(z) + z \cdot \sigma(z)(1 - \sigma(z))$

Strengths

- Smooth and non-monotonic (dips slightly below 0 for small negative z) — empirically this small 'bump' seems to help optimization in very deep networks.
- Unlike ReLU, gradient is never exactly 0, so neurons practically never fully die.
- Found by neural architecture search to outperform ReLU on ImageNet-scale models.

Weaknesses

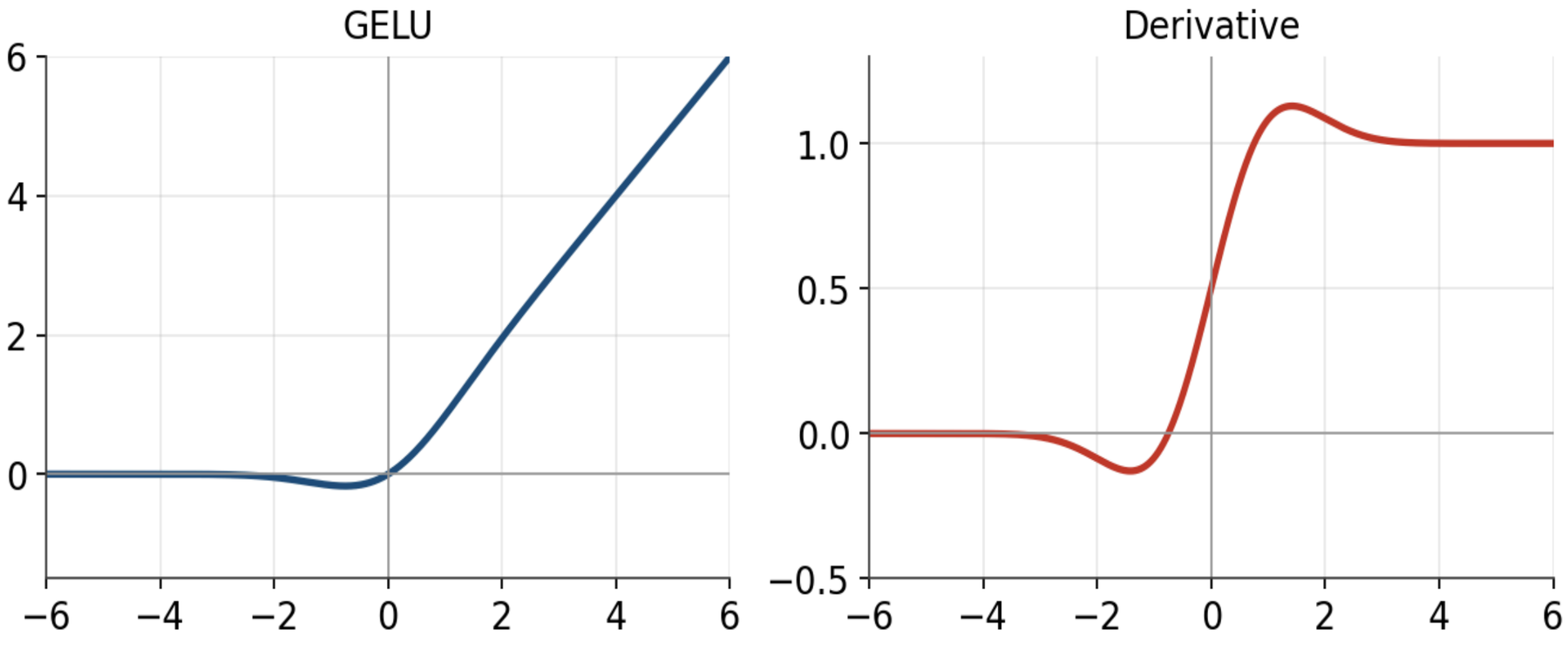
- More expensive to compute than ReLU (requires a sigmoid).
- Benefits are most visible in deep networks (50+ layers) — often negligible in small/shallow ones.

Typically used in: deep CNNs (e.g., EfficientNet) and increasingly in Transformer variants.

4.2 GELU (Gaussian Error Linear Unit)

$$f(z) = z \times \Phi(z) \quad (\Phi = \text{standard Gaussian CDF; common approximation shown in the graph})$$

Range: $\approx (-0.17, \infty)$



Derivative: computed numerically — smooth everywhere

Strengths

- Weights the input by how likely it is to be 'kept' under a Gaussian, giving a smooth, probabilistic version of ReLU.
- Smooth curvature near zero avoids ReLU's hard kink, which empirically improves optimization in very deep stacks.

Weaknesses

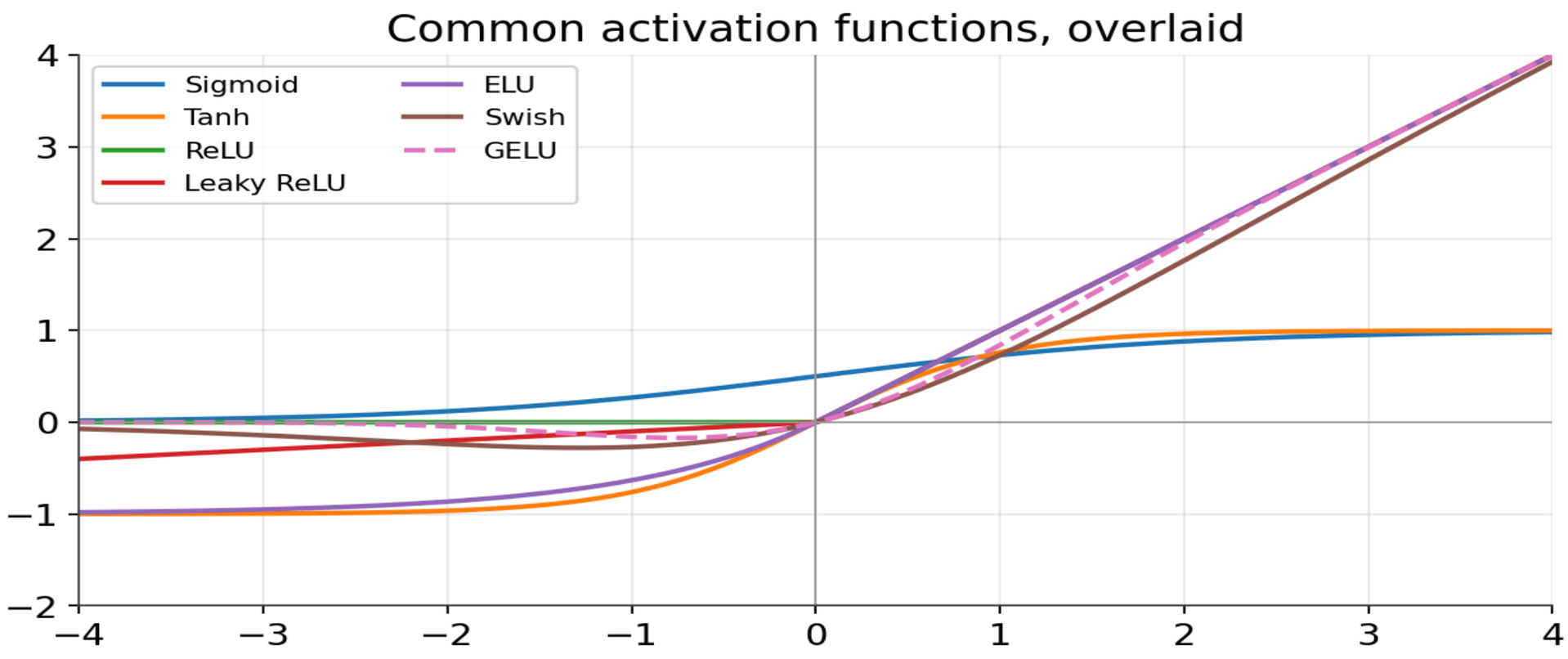
- More expensive to compute than ReLU (uses erf or a tanh-based approximation).
- Marginal benefit over ReLU/Swish is architecture- and scale-dependent.

Typically used in: the de facto standard hidden-layer activation in Transformer architectures (BERT, GPT-family, ViT, and most modern LLMs).

> ***Note:*** *GELU is the activation you are most likely to encounter if you go on to read the internals of a modern Transformer / large language model.*

4.3 Visual Comparison

The plot below overlays the functions covered so far. Notice how the saturating functions (sigmoid, tanh) flatten at both ends, ReLU-family functions are linear (or nearly linear) for $z > 0$, and Swish/GELU add a smooth dip just below zero instead of a hard cutoff.


Common activation functions, overlaid
Sigmoid
Tanh
ReLU
Leaky ReLU
ELU
Swish
GELU
4
3
2
1
0
−1
−2
−4
−3
−2
−1
0
1
2
3
4

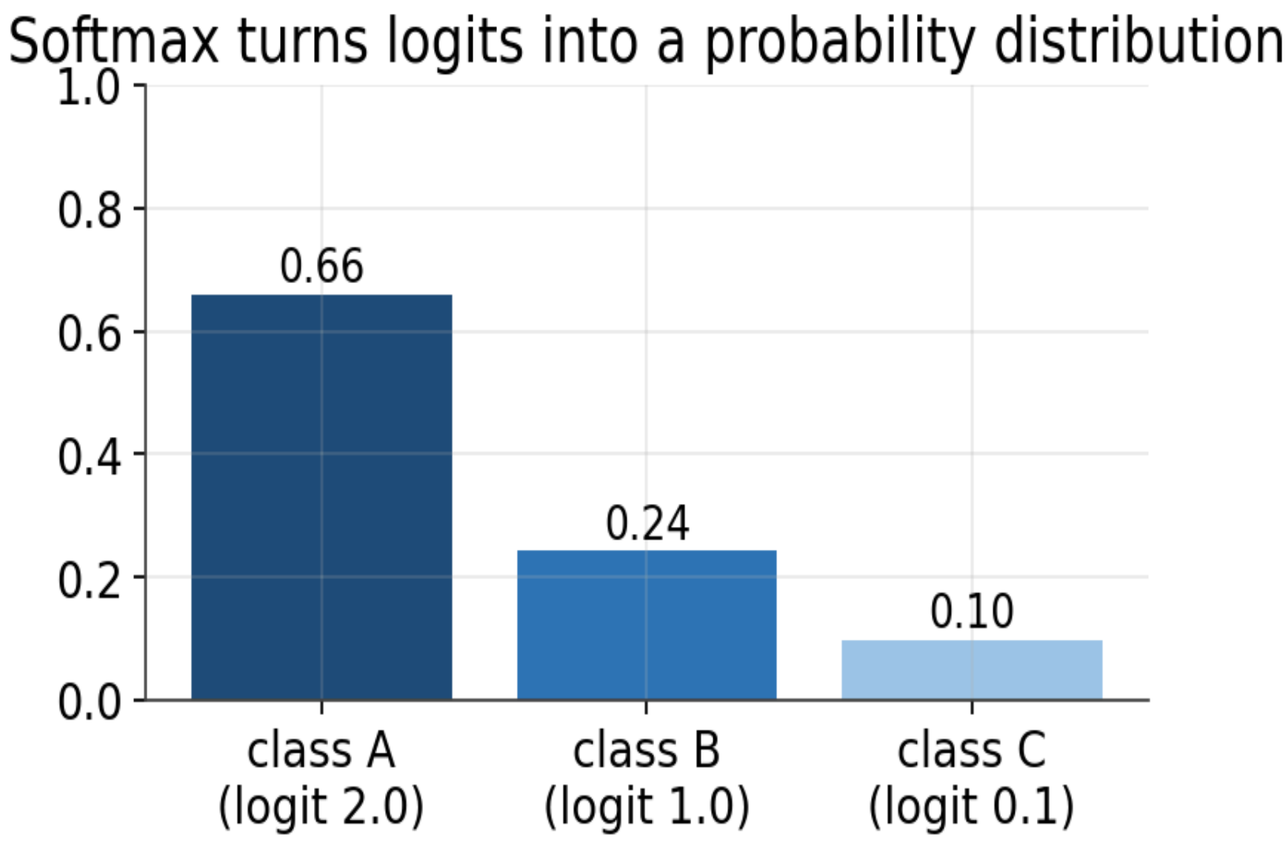
5. Output-Layer Activations

The activation used in the final layer is chosen based on the task, not for gradient-flow reasons — it shapes the output into whatever form the loss function expects.

5.1 Softmax

For multi-class classification with K mutually exclusive classes, softmax converts a vector of raw scores (logits) $z = [z_1, ..., z_K]$ into a probability distribution:

$$\text{softmax}(z)_i = e^{z_i} / \Sigma_j e^{z_j}$$



- Outputs are all positive and sum to exactly 1 — a valid probability distribution over classes.
- Used with the cross-entropy loss for standard multi-class classification (e.g., the final layer of an image classifier choosing among 1,000 categories).
- Not used in hidden layers — it's a whole-vector operation (each output depends on every input), unlike the neuron-wise functions above.

5.2 Quick reference: which output activation for which task?

Task	Output activation	Typical loss
Binary classification	Sigmoid	Binary cross-entropy
Multi-class, single label	Softmax	Categorical cross-entropy
Multi-label (independent classes)	Sigmoid (one per label)	Binary cross-entropy per label
Regression (unbounded output)	Linear / none	MSE / MAE
Regression (output must be positive)	ReLU / Softplus	MSE / MAE

6. The Vanishing / Exploding Gradient Problem

During backpropagation, the gradient reaching an early layer is a product of many terms — one derivative of the activation function per layer between there and the output (this is exactly the chain-rule pattern from the backpropagation worked example).

- If each of those derivatives is consistently < 1 (as with sigmoid, whose derivative maxes out at 0.25), the product shrinks geometrically with depth — the gradient 'vanishes,' and early layers stop learning. This is why deep networks built purely from sigmoid/tanh layers are hard to train.
- If each derivative is consistently > 1, the product can grow geometrically instead — the gradient 'explodes,' causing unstable, huge weight updates.

ReLU and its variants mitigate vanishing gradients because the derivative is exactly 1 (not <1) for all positive inputs, so it doesn't shrink the signal as it passes through many layers. This — more than any other single change — is why ReLU-family activations enabled training much deeper networks than were previously practical.

> ***Note:*** *Vanishing/exploding gradients are also addressed by other techniques independent of activation choice: careful weight initialization (e.g., He or Xavier initialization), batch/layer normalization, residual (skip) connections, and gradient clipping.*

7. Practical Guidelines for Choosing an Activation Function

Situation	Reasonable default
Hidden layers, general-purpose CNN/MLP	ReLU (start here; it's cheap and works well)
Seeing many 'dead' neurons during training	Leaky ReLU, PReLU, or ELU
Very deep network (dozens+ of layers), esp. Transformer	GELU or Swish
Plain deep MLP without batch norm	SELU (with matching initialization)
Recurrent layers (vanilla RNN, LSTM/GRU gates)	Tanh (state), Sigmoid (gates) — largely fixed by the architecture
Binary classification output	Sigmoid
Multi-class classification output	Softmax
Regression output	Linear (no activation)

In practice: start with ReLU (or GELU if you're building a Transformer-style model) for hidden layers, and choose the output activation from the task-based table in Section 5.2 — then only experiment with alternatives if you observe a specific problem (dead neurons, slow/unstable convergence).

8. Review Questions

- Why does stacking many linear layers without any activation function fail to add representational power, no matter how many layers are used?
- Sketch (or describe) what happens to the gradient flowing backward through 20 stacked sigmoid layers, and explain why.

- A colleague reports that after training, roughly 40% of the ReLU neurons in a hidden layer output exactly 0 for every example in the dataset. What is this phenomenon called, and name two activation functions that would fix it?
- Why is softmax not appropriate as a hidden-layer activation function, even though it is smooth and differentiable?
- For a network predicting a house price (a single unbounded positive number), what would you use as the output-layer activation, and why?

Further Reading

Nair, V. & Hinton, G. (2010). Rectified Linear Units Improve Restricted Boltzmann Machines.

Clevert, D-A., Unterthiner, T. & Hochreiter, S. (2015). Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs).

Klambauer, G. et al. (2017). Self-Normalizing Neural Networks (SELU).

Ramachandran, P., Zoph, B. & Le, Q. (2017). Searching for Activation Functions (Swish).

Hendrycks, D. & Gimpel, K. (2016). Gaussian Error Linear Units (GELUs).